

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 02 -

BALANCEAMENTO DE CARGA NA PRÁTICA

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	25
REFERÊNCIAS.....	26

EMSE

O QUE VEM POR AÍ?

Nesta segunda aula, vamos sair dos fundamentos e mergulhar na parte prática do Balanceamento de Carga com Nginx. Você descobrirá como os grandes sistemas conseguem manter milhares (ou até milhões) de acessos on-line, distribuindo requisições entre vários servidores sem que o usuário perceba. Também vamos explorar mecanismos de Health Checks e Failover, que garantem que seu sistema continue funcionando mesmo quando um dos servidores falha.

Ao longo da aula, teremos **três momentos principais**: **(1) entender como configurar o Nginx** para distribuir requisições de forma eficiente, **(2) simular falhas** e observar como o balanceador reage para manter a disponibilidade e **(3) aplicar monitoramento, testes de carga e troubleshooting** para validar e otimizar o ambiente.

Em cada etapa, você terá hands on com Docker Compose, replicando situações reais que um(a) engenheiro(a) DevOps enfrenta em seu dia a dia. Prepare-se: vamos além da teoria para vivenciar como a alta disponibilidade acontece na prática!

HANDS ON

Nesta aula, você será conduzido(a) por três experimentos práticos que reforçam toda a teoria sobre balanceamento de carga com Nginx. O primeiro passo será configurar um ambiente com múltiplos back-ends em Docker Compose, aplicando na prática os **principais métodos de distribuição** vistos anteriormente: **Round Robin (distribuição uniforme)**, **Least Connections (escolha do servidor menos carregado)** e **IP Hash (persistência de sessão por cliente)**. Essa etapa inicial mostrará como o Nginx coloca em ação diferentes estratégias de balanceamento para garantir eficiência e escalabilidade.

Na sequência, avançaremos para os Health Checks e Failover, configurando parâmetros como `max_fails` e `fail_timeout` no bloco upstream. Simularemos falhas ao parar containers e observaremos o comportamento automático do Nginx redirecionando o tráfego para servidores saudáveis ou para um backend de backup. Essa prática evidenciará a importância da resiliência e da alta disponibilidade em ambientes reais, preparando você para lidar com situações críticas de produção.

Por fim, exploraremos o monitoramento, testes de carga e troubleshooting. Habilitaremos o módulo `stub_status` para expor métricas do Nginx, utilizaremos o Apache Bench (ab) para simular tráfego de usuários concorrentes e analisaremos os logs de acesso e erro para identificar gargalos e problemas comuns, como erros 502 e 504. O objetivo é que você não apenas configure um Load Balancer, mas aprenda a validar, monitorar e diagnosticar ambientes reais, desenvolvendo habilidades essenciais para um engenheiro de DevOps.

Exemplo de execução do Apache Bench durante os testes:

```
ab -n 1000 -c 10 http://localhost/
```

Código-fonte 1 – BASH Exemplo de execução do Apache Bench (ab) em teste de carga
Fonte: Elaborado pelo autor (2025)

O código-fonte e os exemplos práticos de cada exercício estão organizados no [repositório](#) do curso

SAIBA MAIS

Nesta seção de aprofundamento, vamos expandir os horizontes. Vamos além do que foi dito nas videoaulas, explorando nuances, comparativos e melhores práticas que transformarão seu entendimento em expertise. Mergulhe fundo, pois cada conceito aqui é um pilar para a construção de infraestruturas modernas e de alta performance!

Balanceamento de Carga: Fundamentos, Tipos e Estratégias

O Balanceamento de Carga é a espinha dorsal da escalabilidade horizontal. Mas essa orquestração do tráfego não é um conceito monolítico. Existem diferentes abordagens e camadas de atuação que definem a inteligência e o poder de um Load Balancer.

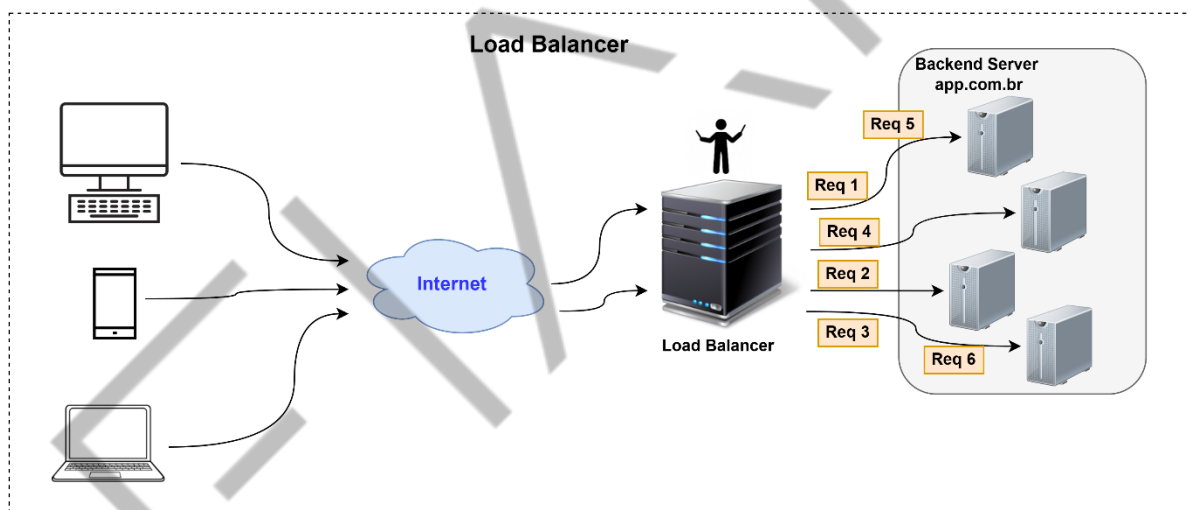


Figura 1 - Como funciona o Balanceamento de Carga em alto nível
Fonte: Elaborado pelo autor (2025)

Hardware vs. Software Load Balancers: A Evolução do Modelo

Historicamente, as soluções de Balanceamento de Carga eram dominadas por Hardware Load Balancers (HLB). Equipamentos físicos de alto desempenho, como os da F5 Networks ou Citrix, eram a norma. Eles ofereciam performance e robustez dedicadas, mas vinham com alto custo, menor flexibilidade e um processo de escalabilidade mais complexo e demorado (pois exigia a compra e instalação de mais hardware).

Com a ascensão da virtualização e, principalmente, da computação em nuvem, os Software Load Balancers (SLB) se tornaram o padrão. Soluções como o Nginx, HAProxy, Envoy, ou os Load Balancers gerenciados por provedores de nuvem (como AWS Application Load Balancer - ALB, Azure Load Balancer, Google Cloud Load Balancing) são softwares que rodam em servidores comuns (commodity hardware). Eles são mais flexíveis, escalam horizontalmente com facilidade (adicionando mais instâncias de software), são mais econômicos e se integram perfeitamente com ambientes de microsserviços e contêineres.

Dica do Professor: hoje, em provedores de nuvem, quase sempre utilizamos Load Balancers gerenciados (AWS ALB, Azure LB, Google Cloud LB), que simplificam a operação e trazem escalabilidade automática.

Camadas de Operação: L4 (Transporte) vs. L7 (Aplicação)

A "camada" de um Load Balancer refere-se ao nível do Modelo OSI em que ele opera. Isso determina a profundidade da inspeção do tráfego e, conseqüentemente, a inteligência e as funcionalidades que ele pode oferecer.

Balanceamento de Camada 4 (L4 - Camada de Transporte):

- **Atuação:** opera na camada de transporte (TCP/UDP). O Load Balancer toma decisões de roteamento baseadas apenas nas informações da camada de rede: IP de origem/destino e portas. Ele não "vê" o conteúdo da requisição HTTP ou HTTPS.
- **Funcionamento:** estabelece uma conexão TCP com o cliente e, em seguida, uma segunda conexão TCP independente com o servidor backend. O tráfego é apenas "encaminhado" sem inspeção profunda.
- **Vantagens:** extremamente rápido e com baixo overhead de processamento, pois não precisa analisar o conteúdo dos pacotes. Ideal para cenários de alta vazão e baixo processamento por requisição.
- **Limitações:** menos "inteligente". Não consegue rotear com base em cabeçalhos HTTP, cookies, caminhos de URL ou conteúdo da requisição. Não pode realizar terminação SSL/TLS, compressão ou cache de conteúdo de aplicação.

- **Uso:** balanceamento de bancos de dados, serviços que usam protocolos não-HTTP, ou onde a inteligência L7 não é necessária e a velocidade é primordial.

Balanceamento de Camada 7 (L7 - Camada de Aplicação):

- **Atuação:** opera na camada de aplicação (HTTP/HTTPS). O Load Balancer consegue inspecionar o conteúdo completo da requisição, incluindo cabeçalhos HTTP (Host, User-Agent), cookies, URLs, parâmetros de query, e até mesmo o corpo da requisição.
- **Funcionamento:** o Load Balancer "termina" a conexão com o cliente (ele próprio atua como o servidor para o cliente), lê e processa a requisição completa na camada de aplicação, toma a decisão de roteamento com base em regras complexas e então abre uma nova conexão com o servidor back-end escolhido para enviar a requisição processada.
- **Vantagens:** muito mais "inteligente" e flexível. Permite roteamento avançado (ex: enviar `api.meusite.com/v1/users` para um pool de microsserviços de usuários e `imagens.meusite.com` para um CDN ou servidor de imagens dedicado). Possibilita terminação SSL/TLS (descarregando essa tarefa dos backends), compressão de conteúdo (Gzip/Brotli), cache de resposta, e integração com recursos de segurança como Web Application Firewalls (WAF).
- **Limitações:** mais custoso em termos de processamento e latência, pois exige que o Load Balancer inspecione e manipule cada requisição em um nível mais profundo.
- **Uso:** a maioria das aplicações web e APIs modernas (e-commerce, SaaS, redes sociais) utiliza Balanceadores L7, sendo o Nginx um exemplo proeminente.

Característica	Load Balancer L4 (Transporte)	Load Balancer L7 (Aplicação)
Camada OSI	Camada 4 (TCP/UDP).	Camada 7 (HTTP/HTTPS).
Informação Inspeccionada	IPs e Portas .	URLs, Headers HTTP, Cookies, Métodos, Corpo da requisição.
Decisão de Roteamento	Baseada em IP/Porta.	Baseada em conteúdo da requisição.
Terminação SSL/TLS	Não (LB transparente ao SSL).	Sim (Comum e recomendada).
Performance	Alta vazão, baixo overhead.	Menor vazão, maior overhead (mais inteligência).
Funcionalidades Avançadas	Limitadas (apenas balanceamento básico).	Cache, Compressão, WAF, A/B testing, Roteamento por URL.
Exemplo de Uso	Balancear bancos de dados, tráfego genérico TCP/UDP.	Aplicações Web (APIs REST, e-commerce, sites dinâmicos).

Tabela 1 - Tabela comparativa: Load Balancer L4 vs L7
Fonte: elaborado pelo autor (2025)

Algoritmos de Distribuição no Nginx: Escolhendo a Melhor Estratégia

O Nginx oferece uma gama de algoritmos para distribuir as requisições, cada um com suas vantagens e cenários de uso ideais. A escolha do algoritmo certo é crucial para otimizar a performance e a experiência do usuário.

Round Robin e o Poder do Peso (weight)

O Round Robin é o algoritmo padrão do Nginx, agindo como um distribuidor justo e sequencial de requisições. No entanto, sua simplicidade pode ser uma limitação em ambientes onde os servidores não são idênticos em capacidade.

Para mitigar isso, o Nginx oferece o **Round Robin Ponderado (Weighted Round Robin)** por meio da diretiva weight. Essa diretiva permite atribuir um "peso" a cada servidor no bloco upstream, indicando a proporção de requisições que ele deve

receber. Servidores mais potentes podem ter um peso maior, garantindo que sejam mais utilizados e otimizando o uso dos recursos da sua infraestrutura.

Exemplo de Configuração com Peso:

```
upstream meus_servidores_backend {  
    server backend_app_1:80 weight=3; # Recebe 3x mais  
    requisições  
    server backend_app_2:80;          # Peso padrão é 1  
    server backend_app_3:80;          # Peso padrão é 1  
}
```

Neste exemplo, para cada 5 requisições, 3 seriam direcionadas para **backend_app_1**, 1 para **backend_app_2** e 1 para **backend_app_3**.

Least Connections: Balanceando a Carga Real

O algoritmo `least_conn` é uma evolução do Round Robin por ser mais "inteligente". Em vez de apenas seguir uma sequência, ele direciona a nova requisição para o servidor back-end que tem o menor número de conexões ativas no momento.

- **Benefícios:** isso é extremamente vantajoso em ambientes onde as requisições podem ter durações de processamento muito variadas (ex: algumas requisições de API são muito rápidas, outras envolvem cálculos complexos e demorados). O `least_conn` garante que requisições longas não "segurem" um servidor desnecessariamente enquanto outros estão ociosos, promovendo um equilíbrio de trabalho e não apenas de contagem de requisições.
- **Quando Usar:** ideal para APIs RESTful, microserviços e aplicações que mantêm conexões persistentes abertas por longos períodos, onde o objetivo é manter todos os servidores igualmente ocupados em termos de recursos, não apenas de requisições recebidas.

IP Hash: Garantindo a Persistência de Sessão (e seus desafios)

O algoritmo ip_hash garante que todas as requisições de um mesmo cliente (identificado pelo seu endereço IP de origem) sejam sempre direcionadas para o **mesmo servidor backend** no pool.

- **Vantagens:** sua principal utilidade é garantir a persistência de sessão (ou "stickiness"). Isso é fundamental para aplicações que armazenam informações de estado do usuário (como itens em um carrinho de compras, dados de autenticação ou preferências) diretamente na memória do servidor, sem um mecanismo de sessão compartilhada entre os backends. Sem o ip_hash, uma requisição subsequente do mesmo usuário poderia ir para um servidor diferente, causando a perda desses dados de sessão.
- **Desafios:** o ip_hash pode levar a um desbalanceamento de carga. Se um grande número de usuários compartilhar o mesmo IP público (comum em redes corporativas, acessos via VPN, ou grandes provedores de internet), todo esse tráfego será direcionado para um único servidor back-end, sobrecarregando-o enquanto outros permanecem ociosos. Além disso, se o servidor ao qual o IP estava "colado" falhar, todas as sessões ativas naquele servidor serão perdidas.
- **Alternativas Modernas à Persistência de Sessão:** em arquiteturas de software contemporâneas, a tendência é construir aplicações Stateless (sem estado). Isso significa que o estado da sessão do usuário não é armazenado na memória de um servidor específico, mas sim em um local centralizado e compartilhado, como:
 - **Bancos de dados de sessão:** utilizando serviços como Redis, Memcached ou até mesmo bancos de dados relacionais para armazenar e recuperar dados de sessão.
 - **Cookies assinados ou Tokens (JWT):** o estado é armazenado de forma segura no próprio navegador do cliente (em um cookie), ou como um token (JSON Web Token) que é enviado a cada requisição e validado pelos servidores. Ao tornar a aplicação stateless, qualquer servidor pode atender a qualquer requisição de qualquer usuário a

qualquer momento, o que simplifica enormemente o balanceamento de carga e aumenta a resiliência do sistema, pois a falha de um servidor não implica em perda de sessão para os usuários.

Porém, em arquiteturas modernas, recomenda-se usar aplicações stateless (sessão em Redis ou JWT). Isso elimina dependência do balanceador para persistência.

Outros Métodos de Balanceamento e Diretivas Avançadas no Nginx

O Nginx é uma ferramenta incrivelmente poderosa e flexível. Além dos métodos principais, ele oferece outras opções e diretivas que permitem um controle granular sobre o tráfego entre o Load Balancer e os back-ends.

Least Time (least_time): disponível no Nginx Plus (versão comercial). Ele não apenas considera o número de conexões ativas, mas também o tempo médio de resposta dos back-ends. O Nginx direciona a requisição para o servidor que tiver o menor tempo de resposta e o menor número de conexões ativas. É ideal para otimização de latência.

Random (random): escolhe um servidor aleatoriamente. Raramente usado em produção para tráfego web, mas pode ter aplicações em testes ou em cenários muito específicos onde a aleatoriedade é desejável.

Hash (hash key): similar ao IP Hash, mas permite criar um hash com base em qualquer variável de texto (ex: cabeçalho HTTP, URL ou combinação de dados). Isso oferece uma "stickiness" mais flexível, mas também pode gerar desbalanceamento se a "chave" usada no hash não for bem distribuída.

Gestão de Conexões: a Diretiva keepalive no upstream. Por padrão, o Nginx pode abrir e fechar uma nova conexão TCP com o servidor back-end para cada requisição recebida. Em ambientes de alto volume, o processo de estabelecimento e encerramento de conexões (o famoso "handshake" TCP e TLS) gera um overhead de CPU e latência. A diretiva keepalive dentro do bloco upstream permite que o Nginx mantenha um certo número de conexões ociosas persistentes abertas para cada servidor backend. Quando uma nova requisição precisa ser enviada para um backend, o Nginx pode reutilizar uma dessas conexões existentes em vez de criar uma nova.

Exemplo:

```
upstream my_app_backends {  
    server backend_app_1:80;  
    server backend_app_2:80;  
    keepalive 32; # Mantém 32 conexões ociosas para cada backend
```

Benefícios: reduz significativamente a latência (pois elimina o handshake de conexão para cada requisição), diminui a carga de CPU tanto no Nginx quanto nos back-ends, e aumenta a vazão total do sistema. É uma otimização de performance crucial.

Controle de Fluxo: timeouts de Proxy. Configurar corretamente os tempos limite (timeouts) é fundamental para a estabilidade e responsividade da sua aplicação. Eles definem por quanto tempo o Nginx deve esperar por certas operações ao se comunicar com os back-ends:

- **proxy_connect_timeout:** tempo máximo para o Nginx estabelecer uma conexão com o backend (default: 60s). Se o backend não aceita a conexão nesse tempo, o Nginx considera uma falha.
- **proxy_send_timeout:** tempo máximo que o Nginx esperará para enviar a requisição completa ao backend após a conexão ser estabelecida (default: 60s).
- **proxy_read_timeout:** tempo máximo que o Nginx esperará por uma resposta completa do backend após enviar a requisição (default: 60s). Este é o timeout mais frequentemente ajustado e monitorado! Se seu backend está processando uma requisição por mais tempo que o proxy_read_timeout, o Nginx encerrará a conexão e retornará um erro 504 Gateway Timeout ao cliente.

Exemplo de Configuração de Timeouts (dentro do location block):

```
location / {  
    proxy_pass http://my_app_backends;  
    proxy_connect_timeout 5s;    # Conexão rápida  
    proxy_send_timeout 10s;      # Envio rápido da requisição
```

```
proxy_read_timeout 30s;      # Espera até 30s pela resposta
do backend
# ... outros headers e configs
}
```

Segurança e Performance: terminação SSL/TLS no Nginx Load Balancer. A terminação SSL/TLS é o processo de descriptografar o tráfego HTTPS. Realizá-la no Load Balancer (Nginx) é uma prática altamente recomendada em quase todas as arquiteturas modernas.

1. O cliente estabelece uma conexão HTTPS criptografada com o Nginx Load Balancer;
2. O Nginx, configurado com o certificado SSL/TLS (e chave privada), descriptografa o tráfego;
3. A requisição (agora em HTTP plano ou re-criptografada, dependendo da configuração de segurança da rede interna) é enviada para o servidor backend;
4. A resposta do backend é recebida pelo Nginx, que a criptografa novamente e a envia de volta ao cliente.

Benefícios:

- **Offloading de CPU:** operações de criptografia e descriptografia são computacionalmente intensivas. Ao realizá-las centralizadamente no Nginx, você "descarrega" essa tarefa dos servidores de aplicação, que podem se concentrar no processamento da lógica de negócio. Isso melhora significativamente o desempenho e a escalabilidade dos back-ends;
- **Centralização da Gestão de Certificados:** a instalação e renovação de certificados SSL/TLS são feitas em um único ponto (o Nginx), simplificando a operação e reduzindo a chance de erros;
- **Otimização de Tráfego Interno:** em muitas arquiteturas, o tráfego entre o Nginx e os back-ends pode ser HTTP (não criptografado) se a rede interna

for considerada segura. Isso elimina o overhead de criptografia e descriptografia contínuas dentro da rede privada.

Conceito	Descrição
SSL Offloading	Transferir a carga de processamento de SSL/TLS (criptografia e descriptografia) dos servidores de aplicação para o Load Balancer.
SSL Passthrough	O Load Balancer apenas repassa o tráfego SSL/TLS diretamente para o backend, sem descriptografá-lo. O back-end é que faz a terminação SSL. Menos comum em L7 e para casos muito específicos.

Tabela 2 – Exemplo de Tabela
Fonte: elaborado pelo autor (2025)

Resiliência e Alta Disponibilidade: Health Checks e Failover

Em sistemas distribuídos, a falha é uma constante. Como disse Werner Vogels, CTO da Amazon, "Tudo falha, o tempo todo." A maturidade de um sistema não está em não falhar, mas em como ele lida com a falha sem impactar o usuário. É aqui que Health Checks e Failover entram em jogo.

Health Checks: Os Olhos do Load Balancer

Um servidor pode estar "no ar" (ligado, respondendo a pings), mas a aplicação web nele pode estar "fora do ar" (travada, com erro de banco de dados, sem memória). Sem um mecanismo inteligente, o Load Balancer continuaria enviando tráfego para esse servidor "doente", causando erros e lentidão para o usuário.

Health Checks são mecanismos que permitem ao Load Balancer **verificar continuamente a saúde e a capacidade de resposta** dos servidores back-end. Se um servidor falha em um certo número de verificações, ele é **automaticamente removido da rotação** de tráfego. Quando ele volta a responder adequadamente, é adicionado de volta.

Benefícios:

- **Proatividade:** detecção de falhas antes que elas afetem todos os usuários;
- **Qualidade de Serviço:** redução drástica de erros e lentidão para o usuário final;
- **Eficiência:** uso apenas de servidores realmente saudáveis e funcionais.

Health Checks Passivos no Nginx (Open Source): o Nginx Open Source implementa Health Checks passivos por meio das diretivas `max_fails` e `fail_timeout` no bloco `upstream`. Eles operam observando as falhas das requisições reais que o Nginx está encaminhando. Se um certo número de requisições falha para um back-end em um determinado período, o Nginx o marca como "doente":

- **max_fails:** número de tentativas de conexão ou requisições que falham antes que o servidor seja considerado offline. Por padrão, 1 (ou seja, uma única falha já tira o servidor da rotação, a menos que a requisição seja muito rápida);
- **fail_timeout:** duas funções: é o período (em segundos) durante o qual as falhas são contadas para o `max_fails`, e também o tempo que o servidor permanecerá marcado como offline. Após esse tempo, o Nginx tentará enviar uma nova requisição (uma "sonda silenciosa") para reavaliar sua saúde.

Exemplo:

```
upstream meus_servidores_backend {  
    server backend_app_1:80 max_fails=3 fail_timeout=15s; # Se  
    3 falhas em 15s, fica offline por 15s  
    server backend_app_2:80 max_fails=1 fail_timeout=5s; # Se 1  
    falha, fica offline por 5s  
}
```

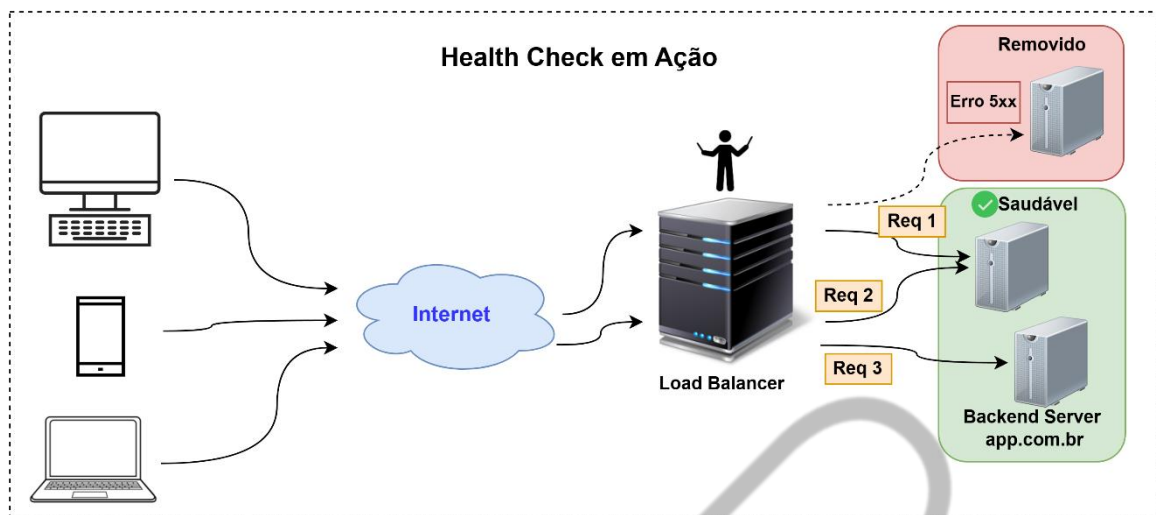


Figura 2 - Health Check Passivo
Fonte: Elaborado pelo autor (2025)

Explicação do Fluxo: o Nginx tenta uma requisição. Se falha, conta uma falha. Se o número de falhas atinge `max_fails` dentro do `fail_timeout`, o back-end é marcado como DOWN e removido da rotação. Após o `fail_timeout` expirar, o Nginx faz uma nova tentativa. Se for bem-sucedida, o back-end é marcado como UP e retorna à rotação.

Health Checks Ativos (Nginx Plus / Módulos de Terceiros): são mais proativos. O Load Balancer **inicia suas próprias requisições de teste** para os back-ends, independentemente do tráfego real dos usuários. Isso permite detectar problemas *antes* que os usuários sejam afetados. Essas verificações podem ser mais granulares, indo além da conectividade básica (ex: verificar a resposta de um endpoint `/health` específico na API, que pode validar a conexão com o banco de dados e outros serviços internos). No Nginx Open Source, essa funcionalidade completa não é nativa; exige o Nginx Plus (versão comercial) ou a compilação de módulos de terceiros.

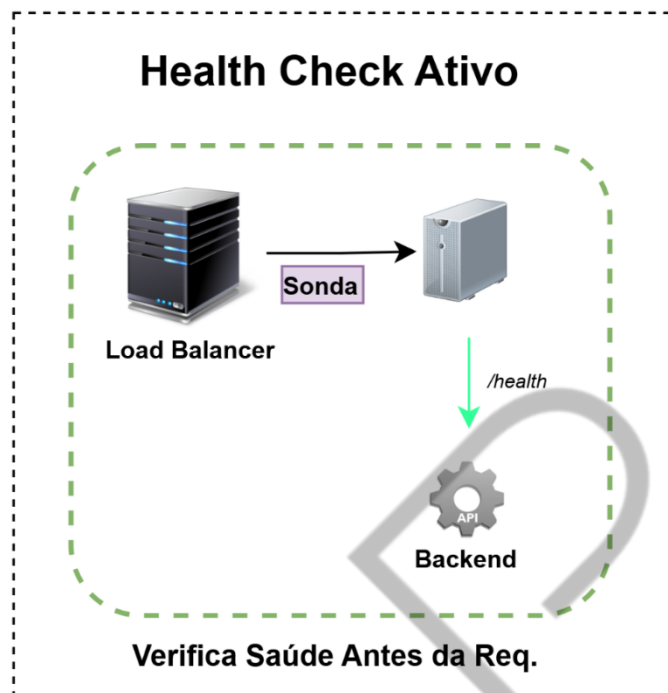


Figura 3 - Health Check Ativo
Fonte: Elaborado pelo autor (2025)

Explicação do Fluxo: o Load Balancer independentemente envia sondas para os back-ends. A resposta da sonda determina se o back-end está OK ou DOWN e o LB age para incluí-lo ou removê-lo da rotação.

Failover: A Reação Automática à Falha

Failover é a capacidade de um sistema de **transferir automaticamente o controle** para um componente de backup ou redundante quando o componente primário falha. É o coração da **Alta Disponibilidade (HA)**, o objetivo de manter o serviço operando continuamente e acessível ao usuário, mesmo diante de falhas de componentes.

Como Alcançar HA com Nginx:

- **Redundância:** ter múltiplas instâncias de cada componente (backends, Nginx Load Balancers);
- **Health Checks:** para detectar as falhas rapidamente;
- **Failover Automático:** para reagir às falhas e reencaminhar o tráfego.

Failover de Backends (com Nginx como Load Balancer): esta é a forma mais comum de failover gerenciada pelo Nginx. Ao detectar que um servidor backend está "doente" via Health Checks (passivos ou ativos), o Nginx o **remove automaticamente da rotação** do upstream. O tráfego passa então a ser direcionado apenas para os servidores saudáveis remanescentes.

A Diretiva backup: o Nginx oferece a diretiva backup no bloco upstream para designar um servidor de reserva. Este servidor só receberá requisições se **todos os outros servidores primários** (aqueles sem a diretiva backup) no mesmo upstream estiverem **offline ou falhando**. É o seu "plano B" automático, útil para cenários de contingência.

Exemplo:

```
upstream meus_servidores_backend {  
    server backend_app_1:80 max_fails=1 fail_timeout=5s;  
    server backend_app_2:80 max_fails=1 fail_timeout=5s;  
    server backend_de_backup:80 backup; # Servidor de contingência  
}
```

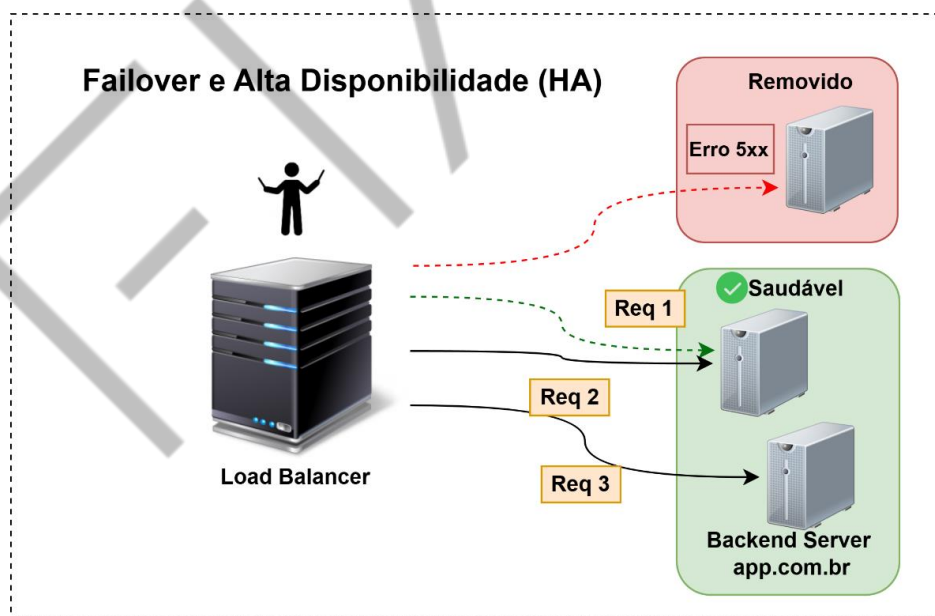


Figura 4 - Failover e Alta Disponibilidade
Fonte: Elaborado pelo autor (2025)

Explicação do Fluxo: o Nginx tenta os back-ends primários. Se nenhum primário estiver disponível, ele recorre ao back-end backup.

Failover do Próprio Load Balancer (Nginx HA): para garantir a Alta Disponibilidade total, o próprio Load Balancer não pode ser um Ponto Único de Falha (SPOF). Ter múltiplos Load Balancers Nginx é a solução, geralmente em um setup ativo-passivo ou ativo-ativo. Isso frequentemente requer ferramentas externas como **Keepalived** (para gerenciar um IP virtual flutuante entre os LBs) ou soluções de Load Balancer gerenciadas por provedores de nuvem (que já lidam com a HA interna).

Operações: Monitoramento, Testes de Carga e Troubleshooting

Ter um sistema balanceado e resiliente é fantástico, mas sem visibilidade e a capacidade de testar e corrigir problemas, você estará operando "às cegas". Aqui, o foco está nos pilares operacionais de qualquer infraestrutura de sucesso.

Monitoramento: Ter Visibilidade é Poder

Monitorar significa ter "olhos" sobre o seu sistema. Não basta que esteja "no ar"; precisamos saber *como* ele está funcionando: se a carga está bem distribuída, se os backends estão performando, se há gargalos silenciosos.

Métricas Nativas do Nginx: ngx_http_stub_status_module. O próprio Nginx oferece um módulo simples, mas vital, para expor métricas básicas de seu estado atual. Ativando um endpoint (geralmente /nginx_status), podemos consultar em tempo real:

- **Active connections:** número total de conexões abertas no momento (clientes e back-ends);
- **server accepts handled requests:** uma linha com três contadores: total de conexões aceitas pelo Nginx, total de conexões tratadas com sucesso, e total de requisições processadas;
- **Reading:** conexões onde o Nginx está lendo o cabeçalho da requisição do cliente;
- **Writing:** conexões onde o Nginx está enviando a resposta ao cliente;
- **Waiting:** conexões que estão ativas, mas ociosas (keep-alive).

Ecosistema de Observabilidade: Prometheus e Grafana

Embora o `stub_status` seja um ponto de partida, para um monitoramento completo e histórico, precisamos de ferramentas que colem, armazenem e visualizem esses dados ao longo do tempo.

- **Prometheus:** é um sistema de monitoramento open-source que "raspa" (scrape) periodicamente o endpoint `/nginx_status` (e outras fontes de métricas) e as armazena em um banco de dados de séries temporais. É excelente para coletar e armazenar métricas. Ele também pode configurar alertas para notificar equipes quando métricas atingem limiares críticos.
- **Grafana:** uma plataforma de visualização de dados open-source que se integra ao Prometheus (e muitas outras fontes de dados) para criar painéis interativos e dinâmicos (dashboards). Transforma dados numéricos em gráficos de fácil compreensão, permitindo identificar tendências, picos e gargalos de forma intuitiva.

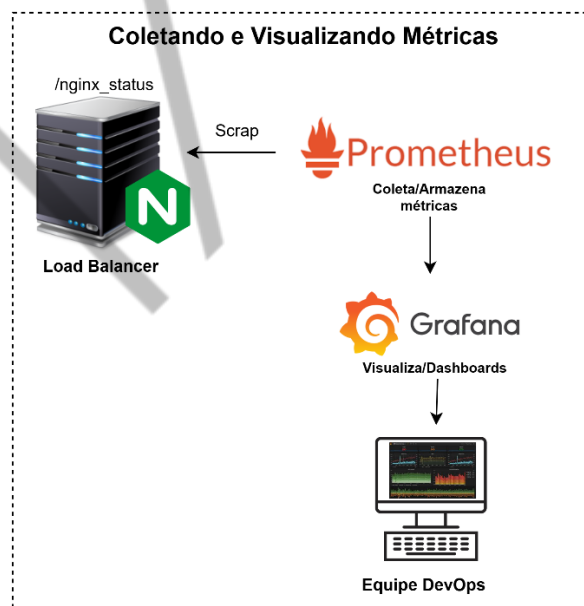


Figura 5 - Failover e Alta Disponibilidade
Fonte: Elaborado pelo autor (2025)

Explicação do Fluxo: o Nginx expõe suas métricas. O Prometheus as coleta e armazena. O Grafana as visualiza em dashboards, e ambos (Prometheus ou Grafana) podem enviar alertas para as equipes.

Testes de Carga: Validando a Capacidade

Monitorar o que está acontecendo é vital, mas testar a capacidade do sistema sob estresse controlado é fundamental para prever e garantir que ele suportará a demanda real.

Importância:

- **Planejamento de Capacidade:** quantos servidores são realmente necessários para X usuários?
- **Identificação de Gargalos:** revelar pontos de falha ou lentidão que só aparecem sob alta demanda.
- **Validação de Alterações:** garantir que novas configurações, código ou infraestrutura não degradem a performance.
- **Preparação para Picos:** simular Black Friday, lançamentos de produtos, campanhas de marketing.

Apache Bench (ab): Uma Ferramenta Simples e Eficaz

O “ab” é uma ferramenta de linha de comando leve e robusta para realizar testes de carga HTTP. É ideal para testes rápidos e validação da performance do Nginx Load Balancer e seus back-ends.

Principais Parâmetros:

- **-n requests:** Número total de requisições a serem feitas;
- **-c concurrency:** Número de requisições a serem feitas **concorrentemente** (simulando múltiplos usuários simultâneos);

Exemplo: `ab -n 1000 -c 10 http://localhost/` (faz 1000 requisições, mantendo 10 requisições abertas ao mesmo tempo).

Interpretando Resultados do ab: as métricas de saída do ab são cruciais para entender o comportamento do sistema sob carga:

- **Requests per second (RPS):** quantas requisições o sistema consegue processar por segundo. **Quanto maior, melhor;**

- **Time per request (TPR):** tempo médio que cada requisição leva para ser completada (considerando apenas uma requisição). **Quanto menor, melhor;**
- **Time per request (across all concurrent requests):** tempo médio para uma requisição ser completada, considerando que várias requisições estão sendo processadas ao mesmo tempo (o que de fato acontece em um ambiente real);
- **Transfer rate:** vazão de dados em KB/s;
- **Percentage of the requests served within a certain time:** distribuição dos tempos de resposta (percentis, ex: 90% das requisições foram atendidas em X ms). Fundamental para entender a experiência da *maioria* dos usuários.

A grande sacada é **combinar o teste de carga com o monitoramento**. Enquanto o ab gera a carga, observe as métricas do Nginx (stub_status) e dos back-ends (CPU, memória, conexões, logs). Se o RPS começa a cair e a CPU de um back-end atinge 100%, você encontrou um gargalo real.

Troubleshooting: A Arte de Resolver Problemas

Mesmo com monitoramento e testes, problemas acontecem. Em sistemas distribuídos, diagnosticar a causa raiz pode ser complexo. A capacidade de identificar e resolver problemas rapidamente é o que diferencia um bom profissional.

Abordagem Sistemática ("Sendo um Detetive Digital")

Comece de Fora para Dentro: comece pelo cliente, depois Nginx (Load Balancer), depois back-ends, depois dependências (banco de dados, APIs externas). Onde a requisição parou ou gerou um erro?

Cheque os Logs (Seu Melhor Amigo)

- **Nginx Access/Error Logs:** verifique se a requisição chegou ao Nginx, quais status codes foram retornados (4xx, 5xx), e se há erros internos.
- **Backend Application Logs:** se a requisição chegou ao back-end, verifique os logs da aplicação para entender como ela processou (ou falhou ao processar) a requisição.

- **Docker Logs:** se usando contêineres, verifique os logs de cada contêiner (`docker logs -f [nome_do_container]`) para erros no nível da plataforma;

Verifique a Configuração do Nginx: use `nginx -t` para validar a sintaxe do seu `nginx.conf` e `nginx -s reload` para recarregar as configurações sem interrupção (se aplicável).

Teste a Conectividade: use ferramentas simples como `curl` ou `telnet` (ou `nc`) para testar a conectividade diretamente do Nginx para os backends. Isso ajuda a isolar problemas de rede ou firewall.

Monitore as Métricas: o `stub_status` do Nginx e seus dashboards no Grafana/Prometheus podem mostrar anomalias (picos de erros, conexões travadas) que indicam a raiz do problema.

Problemas Comuns e Suas Causas Típicas:

Erro 502 Bad Gateway (retornado pelo Nginx):

- **Sintoma:** o Nginx não conseguiu se comunicar com o back-end ou o back-end respondeu com um erro que o Nginx não conseguiu processar (ex: backend caiu, está inacessível, ou a conexão foi recusada);
- **Causas Comuns:** back-end fora do ar (container parado, processo da aplicação travado), firewall bloqueando a porta, back-end não inicializou corretamente, erro na configuração `proxy_pass` (IP/porta incorreta);
- **Verificação:** `docker ps` (o container do backend está rodando?), `docker logs -f [backend]` (logs da aplicação no backend), `curl http://[IP_do_backend]:[porta_do_backend]` (testar acesso direto do Nginx ao backend), `nginx error.log`;
- **Solução:** iniciar/reiniciar o back-end, corrigir a configuração de rede/firewall, otimizar a inicialização do backend.

Erro 504 Gateway Timeout (retornado pelo Nginx):

- **Sintoma:** o Nginx conseguiu se comunicar com o back-end, mas o back-end demorou mais que o tempo limite (`proxy_read_timeout`) configurado no Nginx para enviar a resposta completa;

- **Causas Comuns:** aplicação do backend está lenta, consultas de banco de dados demoradas, processamento pesado, deadlock, ou `proxy_read_timeout` configurado para um valor muito baixo;
- **Verificação:** logs da aplicação no backend (há erros de desempenho?), logs do Nginx (qual o `proxy_read_timeout` atual?), métricas de tempo de resposta dos backends;
- **Solução:** otimizar a performance da aplicação no back-end, otimizar consultas ao banco de dados, escalar verticalmente ou horizontalmente o backend, ou aumentar `proxy_read_timeout` (com cautela, pois pode mascarar problemas de performance).

Requisições não são Balanceadas como Esperado:

- **Sintoma:** um back-end está sobrecarregado enquanto outros estão ociosos, ou as requisições não se alternam como esperado (ex: Round Robin não funciona);
- **Causas Comuns:** upstream mal configurado, server com weight indevido, `ip_hash` causando desbalanceamento (muitos IPs se mapeando para um único servidor), Health Checks passivos removendo servidores indevidamente, ou problemas de rede;
- **Verificação:** recarregar o navegador várias vezes e observar o header X-Back-end-Server, verificar `nginx.conf` (principalmente o bloco upstream e as diretivas de balanceamento), logs do Nginx para ver para onde as requisições estão sendo encaminhadas;
- **Solução:** corrigir `nginx.conf`, rever o método de balanceamento para um mais adequado ao tráfego (ex: `least_conn` em vez de `ip_hash`), ajustar weight.

Ao dominar esses conceitos e ferramentas, você estará não apenas apto(a) a configurar um Load Balancer, mas a operar e manter sistemas complexos em produção, garantindo performance, resiliência e, acima de tudo, a continuidade do negócio. O caminho para se tornar um engenheiro de DevOps de excelência passa por essa profunda compreensão da operação de sistemas distribuídos.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você compreendeu a importância do balanceamento de carga como alicerce da escalabilidade e da alta disponibilidade em sistemas modernos. Explorou os principais algoritmos do Nginx: Round Robin, Least Connections e IP Hash, entendendo seus benefícios, limitações e cenários de aplicação. Aprofundou-se na configuração de health checks e failover, garantindo resiliência diante de falhas de back-ends.

Também conheceu formas de monitorar o Nginx com o módulo `stub_status`, realizou testes de carga com o Apache Bench (`ab`) e analisou métricas de desempenho em situações de estresse. Por fim, desenvolveu práticas de troubleshooting para diagnosticar e resolver problemas comuns, consolidando sua habilidade de projetar e operar ambientes distribuídos de maneira eficiente e confiável.

REFERÊNCIAS

APACHE SOFTWARE FOUNDATION. **Apache HTTP Server Documentation** (v2.4). 2025. Disponível em: <https://httpd.apache.org/docs/2.4/>. Acesso em: 01 set. 2025.

DOCKER. **Docker Compose Documentation**. 2025. Disponível em: <https://docs.docker.com/compose/>. Acesso em: 01 set. 2025.

DOCKER. **Docker Documentation**. 2025. Disponível em: <https://docs.docker.com/>. Acesso em: 01 set. 2025.

GOOGLE CLOUD. **Cloud Load Balancing overview**. 2025. Disponível em: <https://cloud.google.com/load-balancing/docs/load-balancing-overview>. Acesso em: 01 set. 2025.

GRAFANA LABS. **Grafana Documentation**. 2025. Disponível em: <https://grafana.com/docs/>. Acesso em: 01 set. 2025.

GRAHAM, R. **The Secret to 10 Million Concurrent Connections – The Kernel Is the Problem, Not the Solution**. High Scalability, 2013. Disponível em: <http://highscalability.com/blog/2013/5/13/the-secret-to-10-million-concurrent-connections-the-kernel-i.html>. Acesso em: 01 set. 2025.

NETFLIX TECHNOLOGY BLOG. **Chaos Engineering Upgraded**. 2019. Disponível em: <https://techblog.netflix.com/2019/01/chaos-engineering-upgraded.html>. Acesso em: 01 set. 2025. Blog de Tecnologia da Netflix

NGINX. **Nginx Official Documentation**. 2025. Disponível em: <https://nginx.org/en/docs/>. Acesso em: 01 set. 2025.

PROMETHEUS. **Prometheus Documentation**. 2025. Disponível em: <https://prometheus.io/docs/introduction/overview/>. Acesso em: 01 set. 2025.

PALAVRAS-CHAVE

Balanceamento de Carga; Nginx; Alta Disponibilidade.

EMSE

The background is a dark, textured surface with a network of thin, light teal lines and small circular nodes. Some nodes are larger and more prominent, while others are smaller and more numerous. The lines connect the nodes in a complex, web-like pattern, suggesting a digital or technological theme. The overall color palette is dark with teal accents.

POS TECH